Vision Aided Navigation - Exercise 7

Student: Shay Morad

GitHub Repository: https://github.com/ShayMorad/3D-Computer-Vision-SLAM

(Note: Code is located on the main branch. Collaborator invites have been sent to course teachers).

Part 7.1: Detect Loop Closure Candidates

To identify statistically viable loop closures without exhaustively matching every frame against the entire history, a geometric proximity filter was applied using the Mahalanobis distance.

- ***Question:*** *How did you implement the shortest path algorithm?*
- ***Answer:*** I implemented Dijkstra's algorithm operating over a dynamically maintained adjacency list of the pose graph. Instead of querying the GTSAM graph for factors on every iteration, which scales at $O(N^2)$ and causes massive computational bottlenecks, the adjacency list is built once and updated dynamically only when a new loop closure edge is verified.
  To find the "shortest" statistical path, the algorithm minimizes the accumulated geometric uncertainty. The edge weights for Dijkstra were defined as the determinant of the accumulated 6×6 relative covariance matrices along the path. Candidates whose squared Mahalanobis distance $(v^T \Sigma^{-1} v)$ fell below the 6-DOF 99% confidence threshold $(\chi^2 = 16.81)$ were passed to the visual validation stage.

Part 7.2: Visual Validation (Consensus)

A purely mathematical proximity check is insufficient, as the vehicle could be driving on an adjacent parallel street or a bridge overpass. To confirm a loop closure, geometric visual consensus was required.

I routed the candidate frames through the 4-way stereo matching pipeline developed in previous exercises. By triangulating the historical 3D points and matching them temporally to the current frame's 2D left/right keypoints, a custom RANSAC P3P algorithm successfully filtered out visual hallucinations and returned a robust inlier mask. To prevent over-constraining the graph with redundant measurements in stationary or slow-moving areas, a "Loop Cooldown" period of 5 **keyframes** was implemented after any successful closure (this could also be solved dynamically by finding the first and last frames of an area that loops, but a constant was chosen for simplicity).

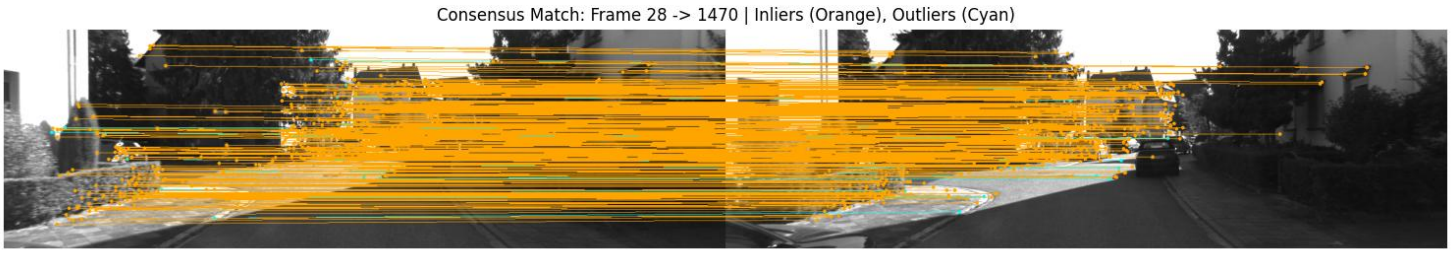


Figure 1: Consensus match visualization between Frame 28 and Frame 1470. Cyan lines indicate outlier matches. Orange lines indicate the surviving inliers that form the rigid geometric consensus proving the loop closure.

Code Reference: ex7.py, Line 119

Part 7.3: Mini-Bundle Adjustment & Covariance Extraction

Because the RANSAC PnP algorithm yields a pose but no uncertainty metric, a localized 2-frame GTSAM factor graph was constructed to extract the highly accurate relative constraint and its marginal covariance.

- ***Question:*** *What is a reasonable initial solution for the Bundle?*
- ***Answer:*** A reasonable initial solution is the relative camera pose generated directly by the RANSAC PnP algorithm in Step 7.2.
- ***Question:*** *How did you extract the appropriate covariance?*
- ***Answer:*** The relative uncertainty was extracted strictly via Information Matrix Conditioning to yield the true conditional covariance $P(c_n|c_i)$. First, a local 2-frame GTSAM factor graph was constructed, anchoring the historical keyframe c_i to the origin to eliminate gauge freedom. After optimizing the robust stereo projection factors, the joint information matrix of the two poses was evaluated. By conditioning the distribution on the anchored frame c_i, the exact relative spatial uncertainty of c_n was isolated.
- **Covariance Inflation (Systemic Error Compensation):**
  Standard visual odometry fundamentally assumes zero systemic error (e.g., perfect lens calibration, zero rolling shutter, perfect lighting). Over thousands of frames, unmodeled micro-errors compound, causing the raw mathematical covariance to become wildly overconfident. To reflect physical reality, I applied Covariance Inflation. The sequential odometry covariances (from Exercise 6) were inflated to prevent the Mahalanobis filter from falsely rejecting real loops due to overconfident dead-reckoning. Conversely, the highly accurate loop closure conditional covariances were scaled by a smaller factor. This ensures the loop closures act as strong "springs"

that tightly pull the trajectory to the true global coordinates without making the graph infinitely rigid.

Part 7.4: Updating the Pose Graph

The extracted relative pose and covariance were injected back into the global pose graph as a BetweenFactorPose3.

To ensure the structural integrity of the map, this global loop closure edge was wrapped in a Cauchy Robust Kernel. Because GTSAM is a least-squares optimizer, a single false-positive loop closure with a massive geometric error would normally warp and destroy the entire trajectory (given the uncertainty is not too high). The Cauchy kernel flattens the penalty for extreme outliers, ensuring that if a bad loop closure bypasses the RANSAC filter, it is dynamically down-weighted to near-zero influence during optimization.

Part 7.5: Final Analysis and Plotting

A Methodological Note on Covariance Scaling, Inflation, and Visualization

Throughout this pipeline, a strict distinction was maintained between the mathematical covariance required by the GTSAM optimizer and the physical covariance rendered in the final visualizations.

- **Backend Mathematical Inflation (The Optimizer):** As noted in Section 7.3, standard visual odometry is mathematically overconfident because it ignores unmodeled systemic errors (e.g., lens distortion, micro-calibration flaws). To prevent the Mahalanobis filter from aggressively rejecting valid loop closures, the dead-reckoning sequential covariances were artificially inflated. Conversely, the geometrically validated loop closures were given a much "stiffer" relative weight (inflated by a smaller scale) to ensure they strongly snapped the trajectory back to the global coordinates.
- **Frontend Mathematical De-Inflation (The True Physics):** If the raw covariances from the optimized graph were plotted directly in Figure 2, the uncertainty ellipses would be artificially inflated by too much, misrepresenting the physical reality of the stereo odometry. Therefore, before rendering the 4 snapshots, the marginal covariances were de-inflated to recover the physical location shape uncertainty of the camera system (but still are inflated more than their true amount that was calculated).
- **Frontend Visual Scaling (The Render):** Even after recovering the true physical covariance, rendering a 1-sigma uncertainty ellipse on a macro-map spanning 600×600 meters results in microscopic dots that are invisible to the human eye. To make the structural collapse of the uncertainty visually apparent, a visual scale factor

was applied exclusively to the geometry of the Matplotlib Ellipse patches. By separating the mathematical inflation from the visual scaling (to draw the 3-sigma confidence bounds), the pipeline successfully optimized the graph while preserving the physical integrity of the final analysis.

Total successful loop closures detected with 5-keyframes loop cooldown: 17
(Total successful loop closures detected with no limitations: more than 80)

Instead of choosing arbitrary frame indices, I deliberately captured snapshots tied to semantic milestones in the optimization process to best illustrate the structural impact of the loop constraints:

- Snapshot 1: The dead-reckoning baseline prior to any loop closures (exhibiting massive uncertainty).
- Snapshot 2: Immediately following the first loop closure to demonstrate the initial global correction.
- Snapshot 3: Midway through the trajectory to show progressive convergence and graph tension.
- Snapshot 4: The final, fully optimized pose graph.

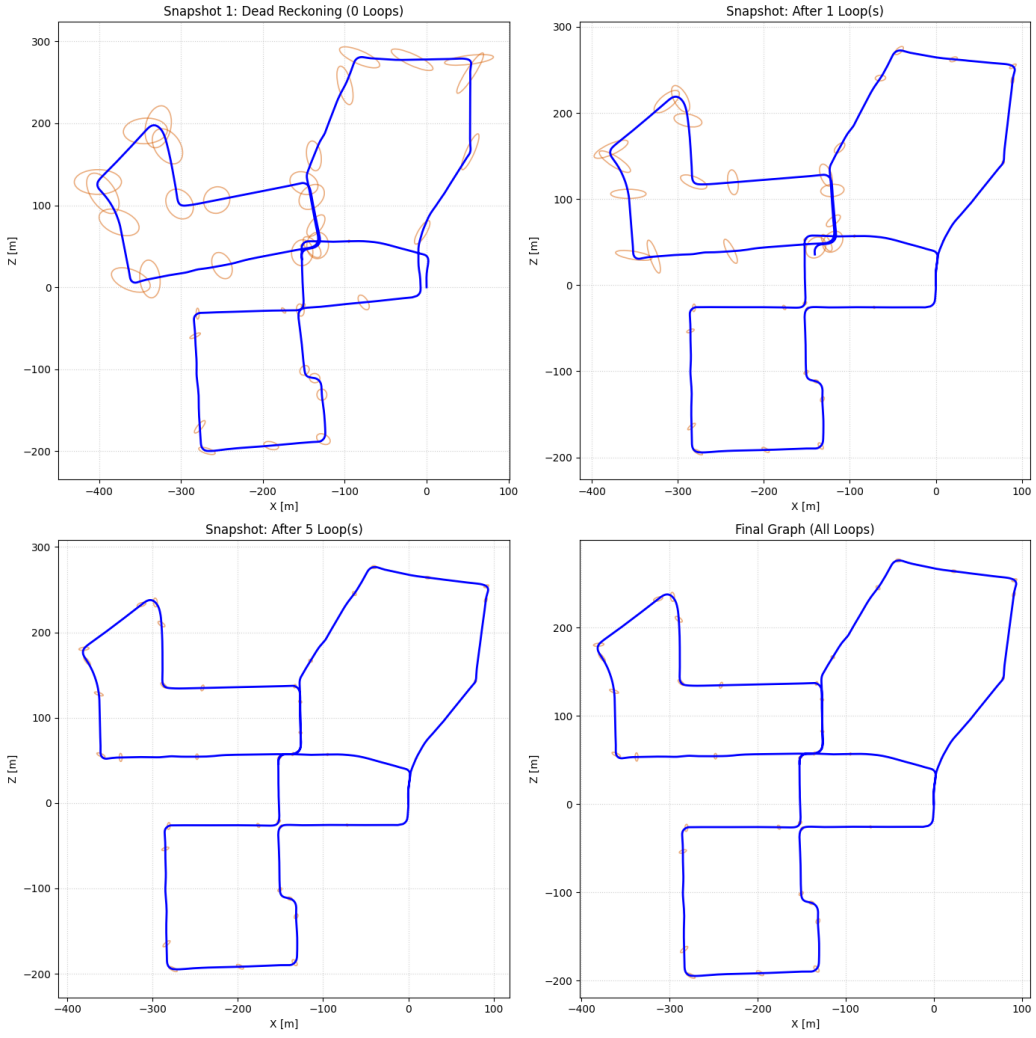


Figure 2: Four snapshots of the pose graph reflecting semantic milestones during the optimization process.

Code Reference: ex7.py, Line 135

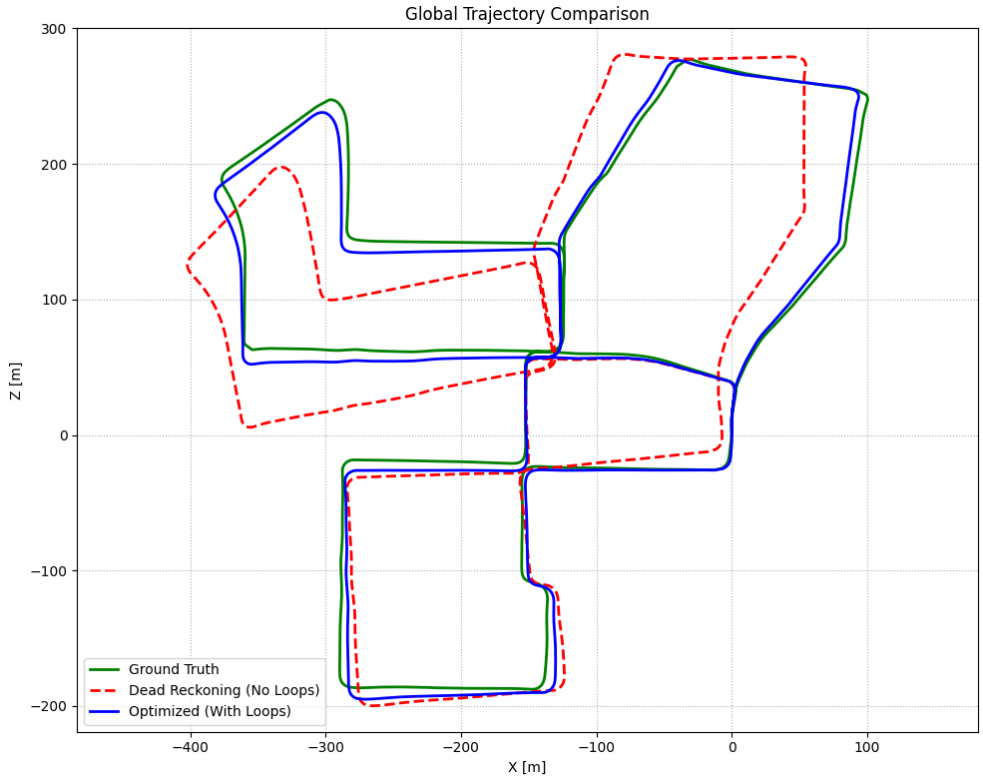


Figure 3: Global trajectory comparison. The dead-reckoning trajectory (Red) exhibits severe integration drift over time. The optimized trajectory (Blue) successfully leverages loop closures to correct this drift, mapping tightly to the true global coordinates (Green).

Code Reference: ex7.py, Line 138

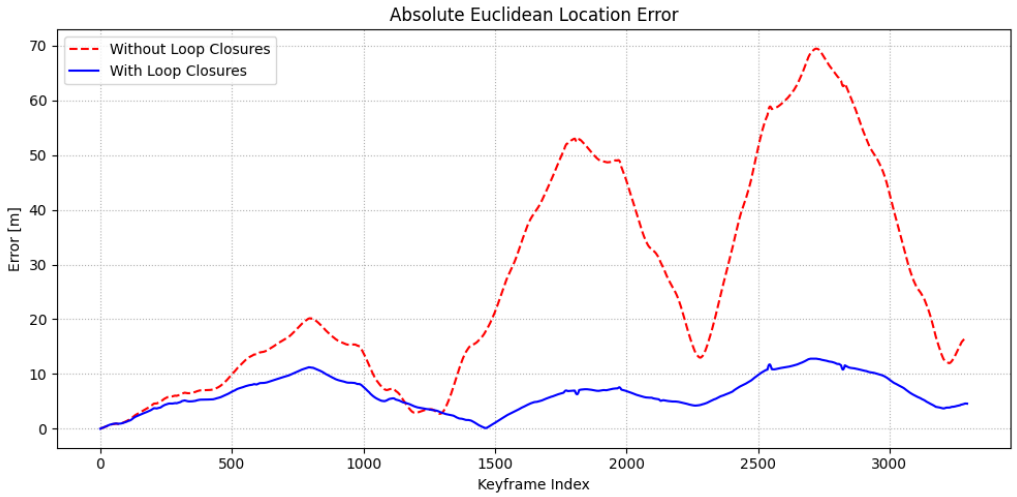


Figure 4Absolute Euclidean Location Error (in meters) over the keyframe sequence. The uncorrected graph accumulates massive positional error over time. Introducing loop closures

significantly reduces this accumulated error: the mean trajectory error decreased from 29.88m to 7.10m, and the drift at the final frame was reduced from 16.42m to just 4.57m.

Code Reference: ex7.py, Line 141

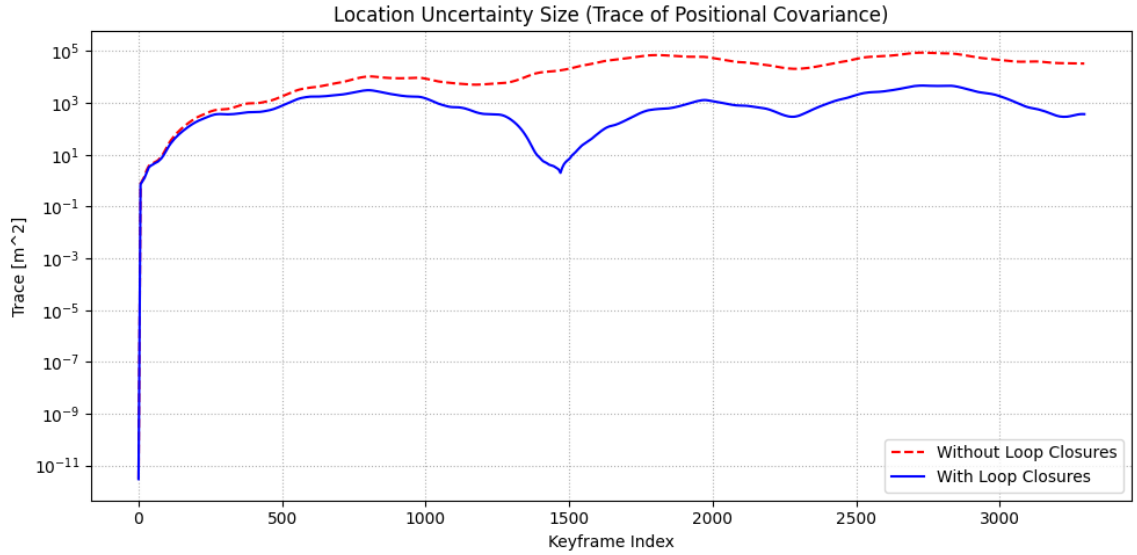


Figure 5: The magnitude of positional uncertainty over time.

Code Reference: ex7.py, Line 147

- ***Question:*** *What measure of uncertainty size did you use?*
- ***Answer:*** I utilized the Trace of the 3×3 positional block extracted from the 6×6 marginal covariance matrix. The Trace represents the total physical variance in the X, Y, and Z directions. I specifically chose the Trace over the Determinant because the Determinant (volume) relies on multiplication; since a ground vehicle's vertical uncertainty is naturally very small, multiplying by it artificially suppresses the reported magnitude of the horizontal drift. The Trace collapses the tensor into a linear sum $(m\text{^}2)$ that safely and accurately reflects the true structural uncertainty. As seen in Figure 5, the Trace drops exponentially whenever a loop closure provides a high-confidence "second opinion" on the vehicle's location.